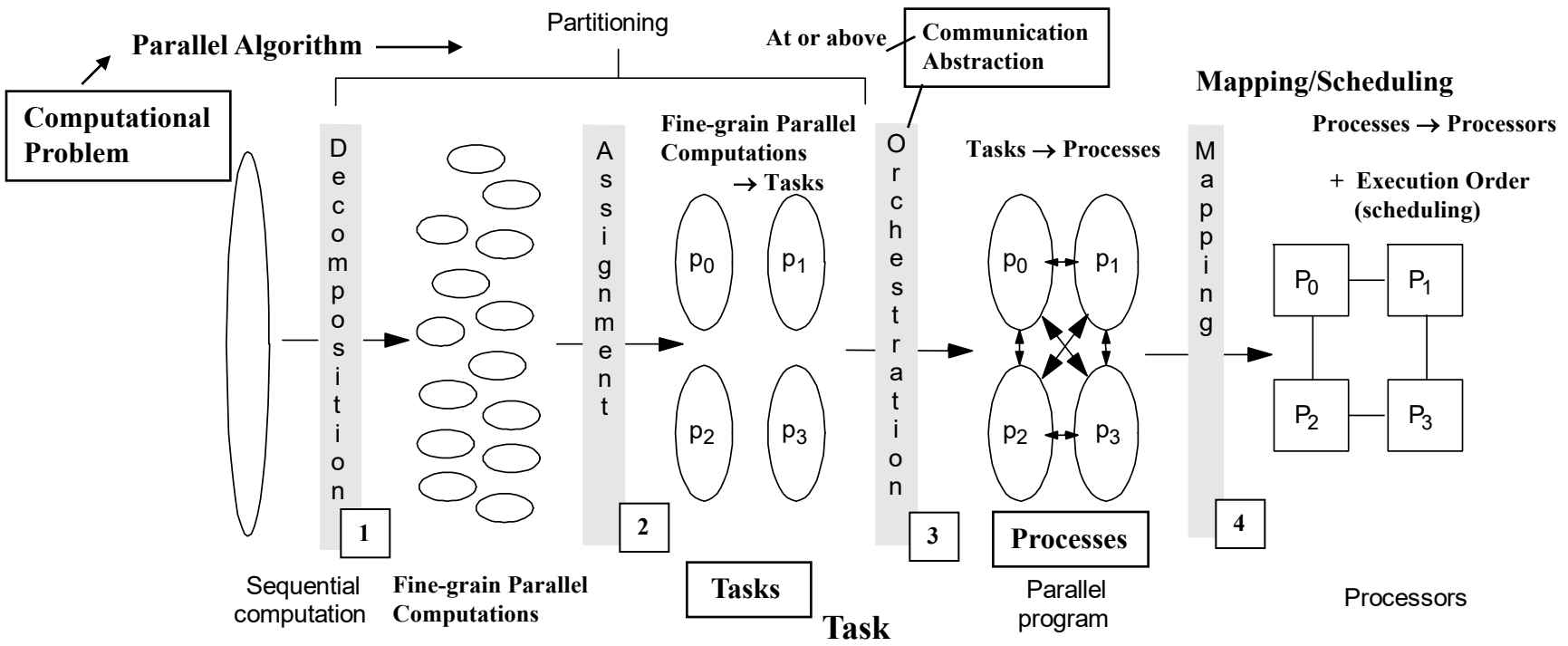
Steps in Creating a Parallel Program



4 steps: Decomposition, Assignment, Orchestration, Mapping

Decomposition = break problem
Assignment = give tasks to processes
Orchestration = coordinate them
Mapping = place processes on processors

Partitioning for Performance

Give approximately equal amounts of useful work to processors.

- **Balancing the workload across tasks/processes:**
  - **Reducing wait time at synchronization points needed to satisfy data dependencies among tasks.** ometimes tasks depend on each other.
- **Reduce Overheads:**

Overhead is extra work required to manage parallelism rather than actually solving the problem.

  - **Reducing interprocess inherent communication.**
  - **Reducing extra work needed to find a good assignment.**

So we want:Good load balance + low communication + low management overhead.

Load Balancing and "Synch Wait Time" Reduction

Limit on speedup:

$$Speedup_{problem}(p) \leq \frac{\text{Sequential Work}}{\underset{\text{(on any processor)}}{Max}\ (\text{Work on any Processor})}$$

- **Work includes computing, data access and other costs.**
- **Not just equal work, but must be busy (computing) at same time to minimize synchronization wait time to satisfy dependencies.**

Four parts to load balancing and reducing synch wait time:

1. **Identify enough concurrency in decomposition.**
2. **Decide how to manage the concurrency.**
3. **Determine the granularity (task grain size) at which to exploit it.**
4. **Reduce serialization and cost of synchronization.**

Identifying Concurrency: Decomposition

Concurrency may be found by:

- **Examining loop structure of sequential algorithm.**
- **Fundamental data dependencies (dependency analysis/graph).**
- **Exploit the understanding of the problem to devise parallel algorithms with more concurrency.**

Parallelism Types:

1 – Data/Domain Parallelism

2- Functional/Task Parallelism:

Identifying Concurrency (continued)

1 - Data Parallelism:

- **Similar parallel operation sequences performed on elements of large data structures**

The same operation is performed on many pieces of data.

- **Such as resulting from parallelization of loops.**

2- Functional Parallelism:

- **Entire large tasks (procedures) with possibly different functionality that can be done in parallel on the same or different data.**
  - **Software Pipelining: Different functions or software stages of the pipeline performed on different data.**

Managing Concurrency: Task Assignment

Goal: ***Obtain an assignment with a good load balance among tasks (and processors in mapping step)***

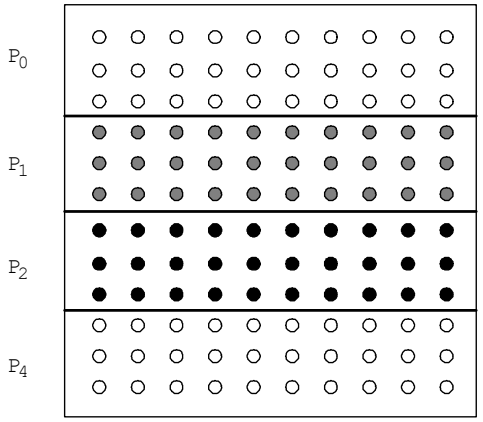
***Static vs Dynamic* Assignment:**

Static Assignment: (e.g equation solver)

i.e 2D Grid Computation

- Algorithmic assignment usually based on input data ; does not change at run time.
- Low run time overhead.

At Compilation Time



Example 2D Ocean Equation Solver

Dynamic Task Assignment:

At Run Time

- Needed when computation not fully predictable.
- Adapt partitioning at run time to balance load on processors.
- Can increase communication cost and reduce data locality.
- Can increase run time task management overheads.

Dynamic Task Assignment/Mapping

Profile-based (semi-static):

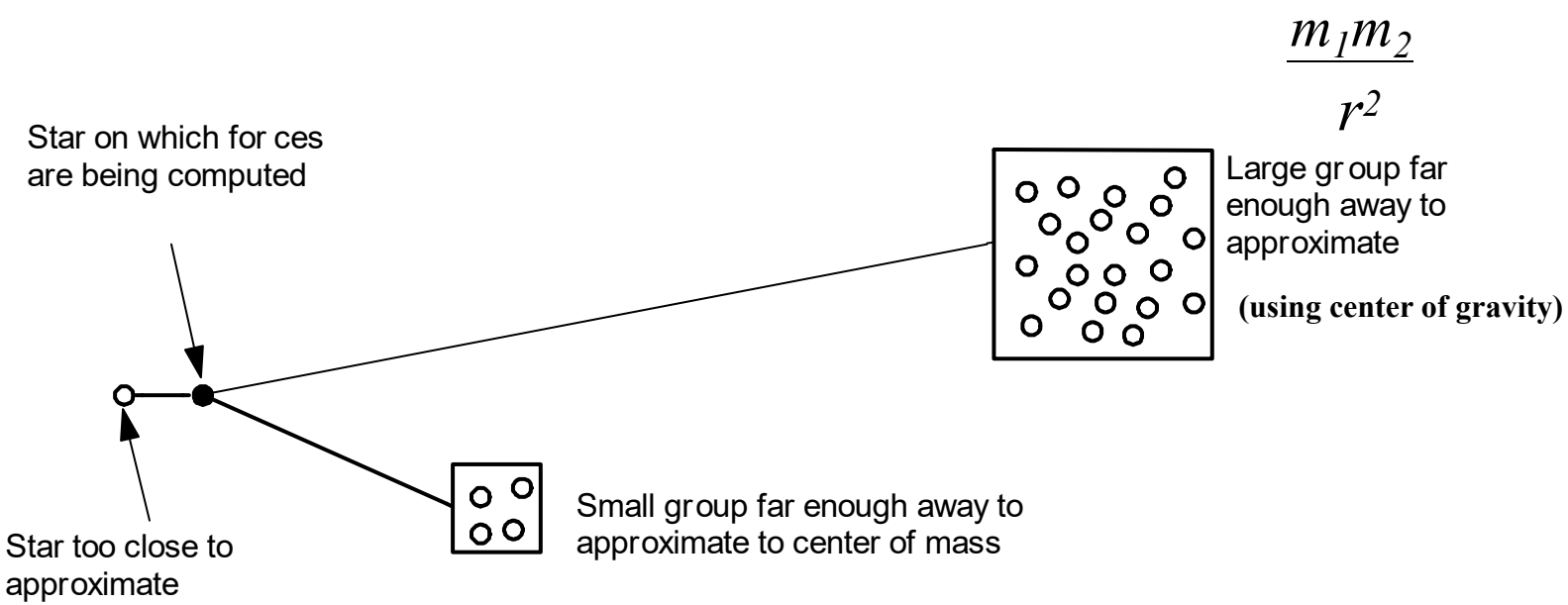
- **Profile (algorithm) work distribution initially at runtime, and repartition dynamically.**
- **Applicable in many computations, e.g. Barnes-Hut, → (*simulating galaxy evolution*) some graphics.**

Dynamic Tasking:

- **Deal with unpredictability in parallel computation, program or environment.**
  - **Computation, communication, and memory system interactions**
  - **Multiprogramming and heterogeneity of processors**
- **Pool (queue) of tasks: Processors take and add tasks to pool until parallel computation is done.**
- **e.g. "self-scheduling" of loop iterations.**

Simulating Galaxy Evolution
(Gravitational N-Body Problem)

- **Simulate the interactions of many stars evolving over time**
- **Hierarchical Methods (e.g. Barnes-Hut) take advantage of force law:** $\frac{m_1 m_2}{r^2}$ ***(center of mass)***



- Many time-steps, plenty of concurrency across stars within one

Gravitational N-Body Problem:
Barnes-Hut Algorithm

- **To parallelize problem: Groups of bodies partitioned among processors. Forces communicated by messages between processors.**
- **Solution: Approximate a cluster of distant bodies as one body with their total mass**
- **This clustering process can be applies recursively.**
- **Barnes_Hut: Uses divide-and-conquer clustering. For 3 dimensions:**
  - **Initially, one cube contains all bodies**
  - **Divide into 8 sub-cubes. (4 parts in two dimensional case).**
    - **If a sub-cube has no bodies, delete it from further consideration.**
    - **If a cube contains more than one body, recursively divide until each cube has one body**
  - **This creates an oct-tree which is very unbalanced in general.**
  - **After the tree has been constructed, the total mass and center of gravity is stored in each cube.**
  - **The force on each body is found by traversing the tree starting at the root stopping at a node when clustering can be used.**
  - **The criterion when to invoke clustering in a cube of size *d x d x d*:**

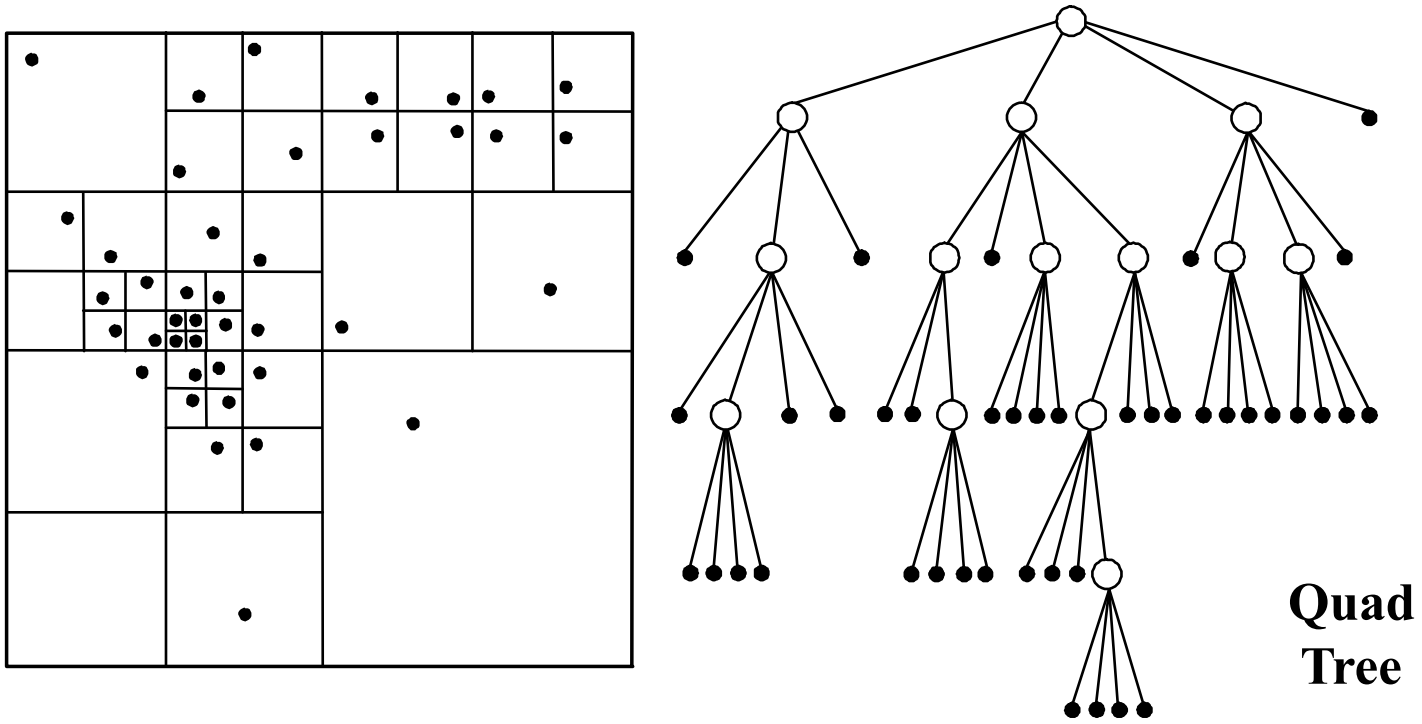
$$r \geq d/\theta$$

r = distance to the center of mass

θ = a constant, 1.0 or less, opening angle

  - **Once the new positions and velocities of all bodies is computed, the process is repeated for each time period requiring the oct-tree to be reconstructed (repartition dynamically)**

Two-Dimensional Barnes-Hut



(a) The spatial domain

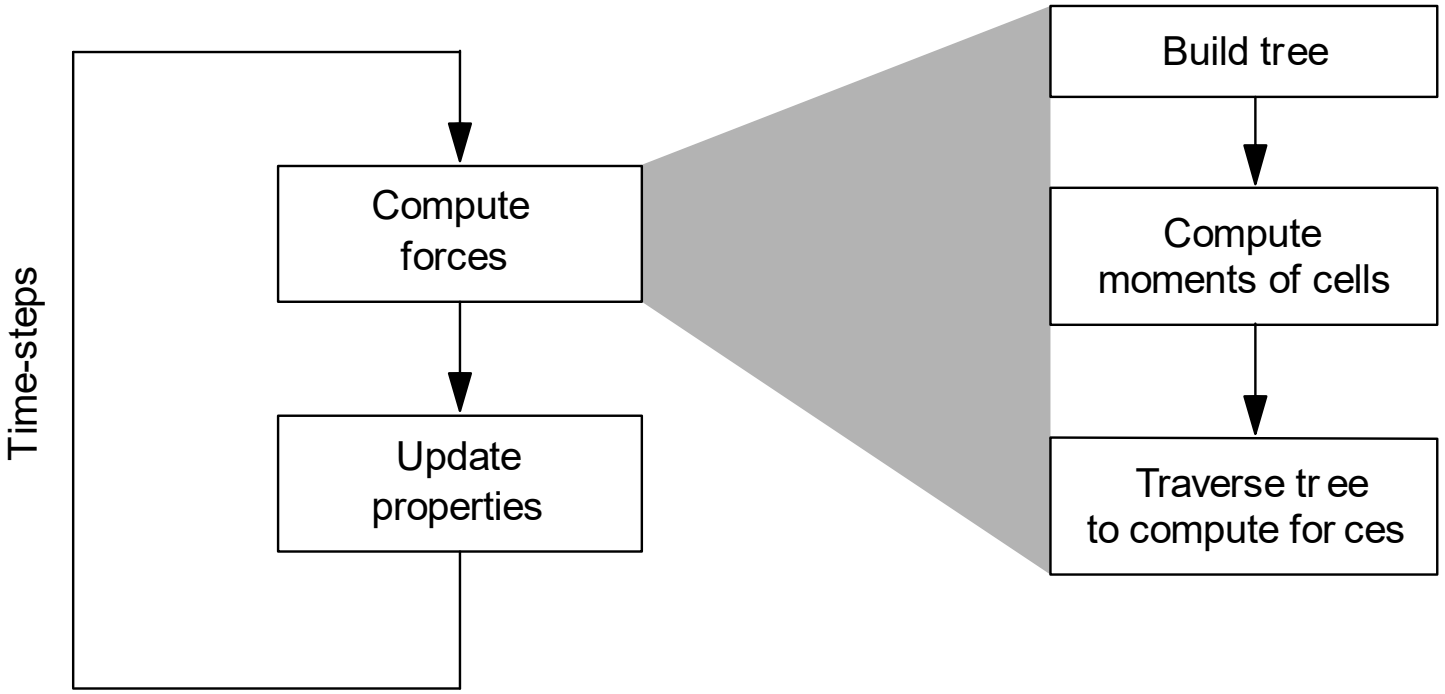
(b) Quadtree representation

Recursive Division of Two-dimensional Space

Locality Goal:

Bodies close together in space should be on same processor

Barnes-Hut Algorithm



Main data structures: array of bodies, of cells, and of pointers to them

- **Each body/cell has several fields: mass, position, pointers to others**
- **pointers are assigned to processes**

The Need For Dynamic Tasking: Rendering Scenes by Ray Tracing

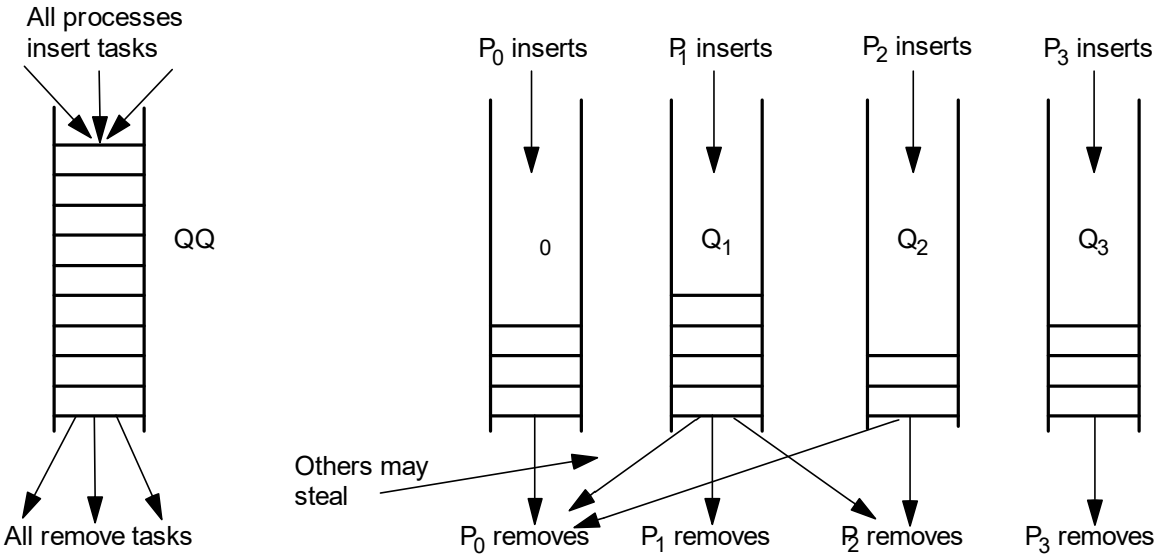
- Shoot rays into a scene through pixels in image plane.
- Follow their paths: To/from light sources/reflected light
  - They bounce around as they strike objects:
    - They generate new rays:
      - Resulting in a ray tree per input ray and thus more computations (tasks).
- Result is color and opacity for that pixel.
- Parallelism across rays.
  - Parallelism here is unpredictable statically.
  - Dynamic tasking needed for load balancing.

Dynamic Tasking with Task Queues

Centralized *vs* distributed queues.

Task stealing with distributed queues.

- Can compromise communication and data locality, and increase synchronization wait time.
- Whom to steal from, how many tasks to steal, ...
- Termination detection (all queues empty).
- Load imbalance possible related to size of task.
  - Many small tasks usually lead to better load balance



Centralized (One) Task Queue

Distributed Task Queues (one per process

Assignment: Determining Task Granularity

Recall that parallel task granularity:

Amount of work or computation associated with a task.

General rule:

- **Coarse-grained** => **Often less load balance**
  Larger/fewer tasks
  less communication and other overheads
- **Fine-grained** => **more overhead; often more**
  Smaller/more tasks
  communication , contention

Communication, contention actually more affected by mapping to processors, not just task size only.

- **Other overheads are also affected by task size too, particularly with dynamic mapping (tasking) using task queues:**
  - **Small tasks -> More Tasks -> More dynamic mapping overheads.**

A task only executes on one processor to which it has been mapped or allocated

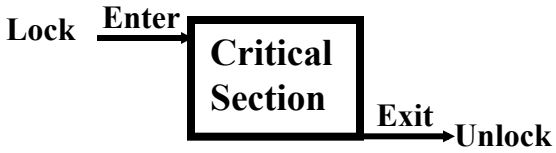
Reducing Serialization/Synch Wait Time

Requires careful assignment and orchestration (and scheduling ?)

Reducing Serialization/Synch wait time in Event synchronization:

- Reduce use of conservative synchronization e.g. :
  - Fine point-to-point synchronization instead of barriers (if possible),
  - or reduce granularity of point-to-point synchronization (specific elements instead of entire data structure).
- But fine-grained synch more difficult to program, more synch operations.

Reducing Serialization in Mutual exclusion:



1 – Separate locks for separate data
  - e.g. locking records in a database instead of locking entire database: lock per process, record, or field
  - Lock per task in task queue, not per queue
  - Finer grain => less contention/serialization, more space, less reuse

2 – Smaller, less frequent critical sections
  - No reading/testing in critical section, only modification
  - e.g. searching for task to dequeue in task queue, building tree etc.

3 – Stagger critical sections in time (on different processors).

i.e critical section entry occur at different times

Implications of Load Balancing/Synch Time Reduction

Extends speedup limit expression to:

$$Speedup_{problem}(p) \leq \underset{\text{(on any processor)}}{\frac{\text{Sequential Work}}{Max\ (\text{Work} + \text{Synch Wait Time})}}$$

Generally load balancing is the responsibility of software

But: Architecture can support task stealing and synch efficiently:

- **F*ine-grained* communication, *low-overhead access* to queues**
  - **Efficient support allows smaller tasks, better load balancing**
- **N*aming* logically shared data in the presence of task stealing**
  - **Need to access data of stolen tasks, esp. multiple-stolen tasks**

=> <u>Hardware shared address space advantageous here</u>

- **Efficient support for point-to-point communication.**
  - **Software layers + hardware (network) support.**

Partitioning for Performance:

Reducing Inherent Communication

Inherent Communication : communication between tasks inherent in the problem/parallel algorithm for a given partitioning/assignment (to tasks)

Measure: ***communication to computation ratio (c-to-c ratio)***

Focus here is on reducing interprocess communication inherent in the problem:

- **Determined by assignment of parallel computations to tasks/processes.**
- **Minimize c-to-c ratio while maintaining a good load balance among tasks/processes.**
- **Actual communication can be greater than inherent communication.**

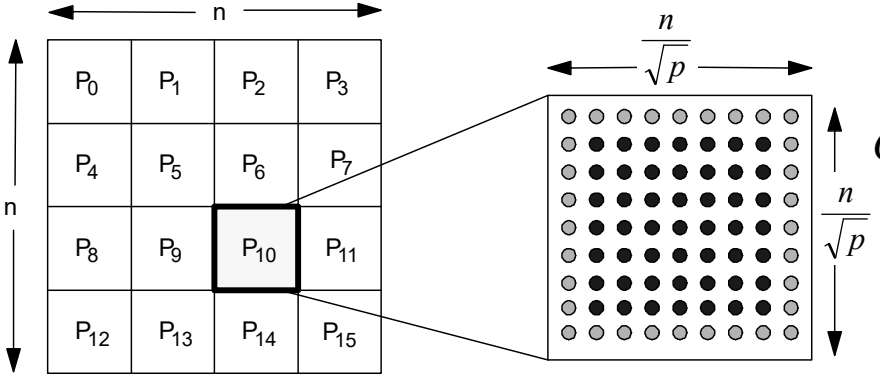
e.g. **As much as possible, assign tasks that access same data to same process (and processor later in mapping).**

Example Assignment/Partitioning Heuristic:

Domain Decomposition

Domain: Physical domain of problem or input data set

- Initially used in data parallel scientific computations such as (Ocean) and pixel-based image processing to obtain a good load balance and c-to-c ratio.
- The task assignment is achieved by decomposing the physical domain or data set of the problem.
- Exploits the local-biased nature of physical problems
  - Information requirements often short-range
  - Or long-range but fall off with distance
- Simple example: Nearest-neighbor 2D grid computation (as in ocean example)



Block Decomposition

$$Communication = \frac{4n}{\sqrt{p}} \quad Computation = \frac{n^2}{p}$$

$$C-to-C = \frac{4 \times \sqrt{p}}{n}$$

comm-to-comp ratio = Perimeter to Area (area to volume in 3-d)

- Depends on *n*, *p*: decreases with *n*, increases with *p*

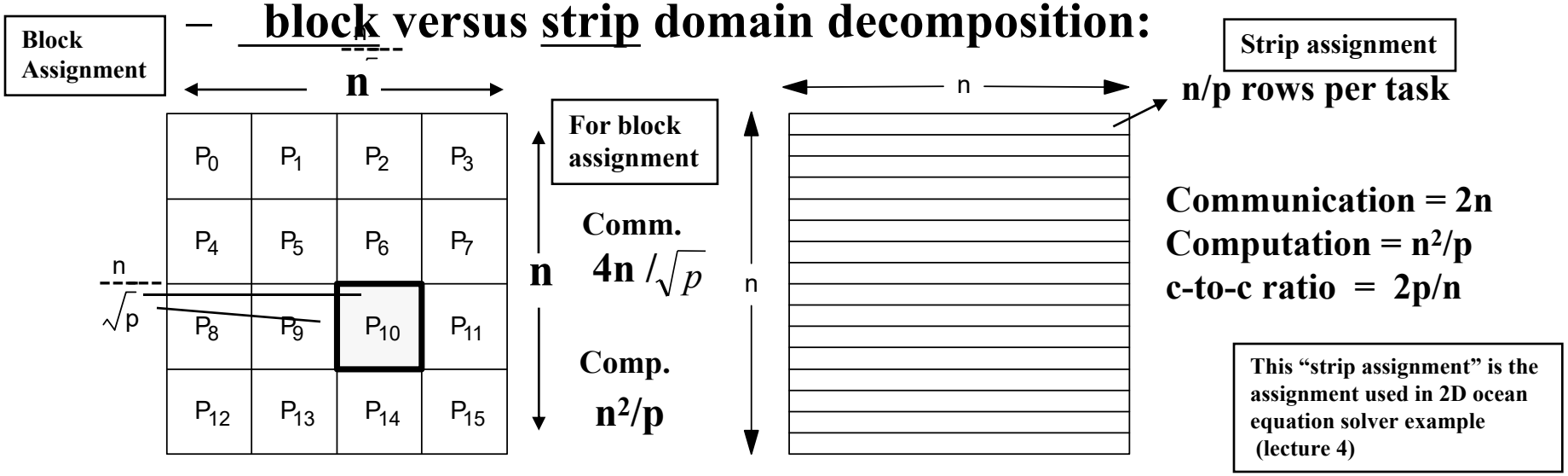
n x n Grid p = Number of tasks/processes here = p= 4 x 4 = 16

Domain Decomposition (continued)

Best domain decomposition depends on information requirements

Nearest neighbor example:

– **block versus strip domain decomposition:**



Block Decomposition **Strip (Group of rows) Decomposition**

Comm-to-comp ratio: $\frac{4 \times \sqrt{p}}{n}$ for block, $\frac{2 \times p}{n}$ for strip

Which C-to-C ratio is better?

Application dependent: strip may be better in some cases

Often n >> p

Implications of Communication

- **Architects must examine application latency/bandwidth needs**
- **If denominator in c-to-c is computation execution time, ratio gives average BW needs per task.**
- **If denominator in c-to-c is operation count, gives extremes in impact of latency and bandwidth**
  - **Latency: assume no latency hiding.**
  - **Bandwidth: assume all latency hidden.**
  - **Reality is somewhere in between.**
- **Actual impact of communication depends on structure and cost as well:**

$$\text{Speedup} \leq \frac{\text{Sequential Work}}{\underset{\textit{(on any processor)}}{\textit{Max}}\,(\text{Work} + \text{Synch Wait Time} + \text{Comm Cost})}$$

→ **Need to keep communication balanced across processors as well.**

c-to-c = communication to computation ratio